Project Findings & Methods Log: Robust and Explainable Classical ML for Tomato Leaf Disease

Oct 13, 2026 · @Manahil Kamran

1. Project overview

This project measures how well lightweight classical models (hand-crafted color, texture and shape features feeding a Random Forest) cope with real-field variation in tomato leaf photos, and which features stay trustworthy when conditions change. This log records what we did, what we found, what went wrong and what limits the conclusions, so each point can be lifted into the mid-semester report and the final paper.

Course and team. CS-871 Machine Learning, MS(AI), NUST SEECS, Fall 2026. Hamna Sadia Rizwan (583282), Maria Ilyas Malik (624045), Manahil Kamran (582413).

Research questions (from the synopsis).

1. How well do simple classical models cope with the visual variation found in real tomato fields?
2. Which image features can still be trusted when lighting, contrast or background change?

Objectives (from the synopsis).

1. Build a baseline on color, texture and shape features with Random Forest as the main model, and SVM and Logistic Regression only as a brief check.
2. Measure the performance loss under illumination/contrast and background perturbations.
3. Run a feature-group ablation (color, texture, shape, combined) to see which cues stay reliable.
4. Test how stable feature importance is across folds and perturbed inputs.

Deliverables and how they are graded (from the term-project brief).

Deliverable	Due	Marks	What is graded
Synopsis (2 pages)	Sep 25, 2026	10	Submitted. Motivation, objectives, dataset exploration, metrics
Mid-semester report	Nov 13, 2026, 23:59 PST	10	Literature survey, methodology tied to course concepts, preliminary results, challenges handled honestly
Final term paper	Dec 24, 2026, 23:59 PST	20	IEEE format, 5-6 pages. Abstract, novelty, results and error analysis, limitations

The brief asks the mid-semester report to include revisions to the original plan with reasoning. Section 3 records the main one: the dataset turned out to be smaller than the synopsis stated.

2. Dataset

We use the Tomato Leaf Disease Classification Dataset in Pakistan (Malik, Ali and Ali, Mendeley Data v2, DOI 10.17632/3mbnb82mxd.2, February 2026), specifically its `Dataset (raw)` folder, which holds 830 original photos. The 7,200 images quoted in the dataset description and in our synopsis belong to the augmented version.

What the dataset documentation states.

Property	Value
Collection	Real agricultural fields in Pakistan, natural daylight, uncontrolled backgrounds
Capture device	Smartphone camera
Labeling	Manual, based on visible symptoms
Format and resolution	JPG, stated as uniform resolution for all images
Classes (6)	Early Blight, Late Blight, Septoria Leaf Spot, Leaf Mold, Yellow Leaf Curl Virus, Healthy
Stated total	7,200 images
License	CC BY 4.0

What the folders actually contain (from the Drive folder and the audit).

Folder	Content	Used?
`Dataset (raw)`	One sub-folder per class, no train/val/test split, random file names. 830 images found	Yes, main data
`Augmented Dataset`	`train`, `val`, `test`, each with one folder per class. Files named like `100_46341_aug_5aug62.jpg` and `100_46341aug1.jpg`, i.e. derived copies of a source photo	No (see section 3)
`Dataset_Description.pdf`, `License.txt`, `README.md`	Documentation	Reference

Class folder names (raw folder, as spelled by the authors) and the label we assign.

Raw folder	Label used in code
`Tomato_Early_blight_leaf`	early_blight
`Tomato_leaf_late_blight`	late_blight
`Tomato_Septoria_leaf_spot`	septoria_leaf_spot

Raw folder	Label used in code
`Tomato_mold_leaf`	leaf_mold
`Tomato_leaf_yellow_curl_virus`	yellow_leaf_curl_virus
`Tomato_Healthy_leaf`	healthy

The augmented folders spell two names differently (`Tomato_septora_leaf_spot`, `Tomato_Early_Blight`), so the class matcher accepts both spellings.

Why this dataset fits the question. The variability we want to study (changing light, busy backgrounds, partly visible symptoms) is already in the photos, so perturbations are layered on top of realistic images instead of on clean lab shots. No published benchmark exists for it yet (checked at synopsis time), so our numbers serve as a first baseline, not a replication.

3. Findings so far

The dataset holds 830 original photos, not 7,200, so the synopsis assumptions about size, balance and split no longer hold and the plan has been revised. The numbers below come from our first real audit run (`00_audit_dataset.py --hash`) on `Dataset (raw)` on Oct 13, 2026.

Audit measure	Value
Images found in `Dataset (raw)`	830
Classes recognised	6
Ignored files or unmatched folders	None reported
Pre-made train/val/test folders	None
Files marked as augmented (by name)	0.0%
Near-duplicate pairs found	55
Of which pair two different classes	9
Independent photos after merging copies	785 (from 830)
Largest group of copies	3 images

Findings.

1. **Real size.** 830 originals versus 7,200 stated. 7,200 / 830 is about 8.7 augmented images per original, which is consistent with the augmented folder being the 7,200-image set. We have not yet audited the augmented folder to confirm its count.
2. **Synopsis assumptions that no longer hold.** The synopsis states 1,200 images per class, a fixed 840/240/120 split per class, balanced classes (so no class weighting) and a 120-image test set per class. These describe the augmented set. For the raw set the per-class counts are not yet known, the split is our own, and the test set will be roughly 125 images in total. This is a plan revision to report, with this reasoning.
3. **Copies inside the raw set.** 45 of 830 images (5.4%) are near-copies of another image (resized, recompressed, flipped or slightly re-lit). We merge them into one group so they cannot sit on both sides of the split.
4. **Possible label noise.** 9 of the 55 near-duplicate pairs carry two different class labels. Either the same photo is filed under two diseases or the matcher paired two similar-looking photos. Each pair needs a visual check; the count is worth one sentence in the paper.
5. **Augmented set risk (not yet quantified on real data).** Augmented files share a stem with their source photo (`100_46341` appears in the raw folder and as `100_46341aug1.jpg`). If the official train/val/test split was made after augmenting, siblings of one photo can sit in train and test and inflate scores. Its test images also already contain brightness, flip and rotation changes, which confounds a perturbation study. We chose raw to avoid both.
6. **Provenance questions from file names.** In the one raw folder we looked at (Early Blight, 8 visible files) some names look like phone camera-roll files (`_storage_emulated_0_DCIM_Camera_IMG_20251102_...`), while others look like a different camera or web source (`20080526-_DSC4211.jpg`, `1576389-early%20blight%20symptoms.jpg`). The documentation says smartphone field photos at uniform resolution. This is an observation from a handful of names, not a conclusion: check it with image sizes (stage 02 writes `eda_image_sizes.csv`) and EXIF data.
7. **Likely same-plant bursts.** Camera-roll names with close timestamps (`IMG_20251102_155855`, `_161045`, `_161217`) suggest several shots of one plant in one session. These are different photos, so the visual grouping will not merge them. They can still make test images easier than a truly new field would be. See limitations.

4. Data preparation

We re-split the 830 photos ourselves into 70% train, 15% validation and 15% test, per class, keeping near-copies of one photo together, so that test scores are not inflated by seeing the same photo in training.

Steps (all in `src/data.py`, `src/duplicates.py`, run by `scripts/01_prepare_data.py`).

1. **Scan and label.** Walk `Dataset (raw)`, map each folder name to a class by keyword, record path, label and a source group.
2. **Hash every image.** Resize to 32x32 grey, take a 2-D DCT, keep the low-frequency 8x8 block without the DC term (63 values), and set each bit by comparison with the median. This is a perceptual hash (pHash): JPEG noise and brightness scaling barely move it. Hashes are also computed on the horizontally, vertically and doubly flipped image, so flipped copies match.
3. **Candidate pairs.** Two images are candidates if their hashes differ in at most 10 of 63 bits.
4. **Verify pairs.** A candidate is accepted only if the 32x32 colour thumbnails (after the same flip) have normalised correlation of at least 0.98. This removes false matches between different leaves that merely look alike.
5. **Group.** Accepted pairs of the same class are merged into one group (union-find). Pairs across classes are counted as possible label noise and not merged.
6. **Split by group.** Within each class, groups are shuffled with seed 42 and filled greedily into train (70%), validation (15%) and test (15%) by image count. No group is ever shared between splits.
7. **Cache.** Every image is resized once to 256x256 (area interpolation, aspect ratio not preserved) and stored in one array in manifest order. All later stages read this cache.
8. **Commit the manifest.** `data/manifest.csv` (path, label, split, class id, group, is_aug) is committed once so all three of us use the identical split.

Parameters.

Parameter	Value	Where
Split ratio	70 / 15 / 15 per class	`dataset.group_split`
Seed	42	`seed`
pHash block, bits	8x8, 63 bits	`duplicates.hash_size`

Parameter	Value	Where
Hamming threshold	10 bits	`duplicates.hamming_threshold`
Correlation threshold	0.98	`duplicates.min_correlation`
Cached image size	256x256	`preprocess.image_size`

Safeguards. The code prints a warning if any group exceeds 2% of the data (a sign of chained false matches) and if a class has too few groups for all three splits. It writes `results/grouping_summary.json` with the pair count, groups before and after, and the largest group.

How the method was validated (synthetic test only). We built a fake dataset with the raw folder layout and 72 planted copies (flipped, recompressed at lower quality, slightly brighter). The final method recovered 72 of 72 and kept all 72 pairs in the same split. These images are synthetic, so the thresholds are not tuned on real photos; they need checking against `results/duplicate_examples.png` on the real data.

5. Preprocessing and leaf segmentation

The dataset has no leaf masks, so we segment the leaf with a simple unsupervised colour rule; its quality limits both the shape features and how realistic the background perturbation is.

Method (`src/segment.py`).

1. Convert the 256x256 image to HSV (OpenCV scale: hue 0-179).
2. Keep pixels with saturation at least 40, value at least 40 and hue between 20 and 100 (yellow to green). Brown soil and brown lesions fall outside the hue range.
3. Morphological opening with a 5x5 ellipse and closing with a 15x15 ellipse.
4. Keep connected components at least as large as the larger of 20% of the biggest component and 1% of the image.
5. Fill holes, so lesions fully enclosed by healthy tissue return to the mask.
6. If the mask covers under 5% of the image, treat segmentation as failed and use the whole image.

A slower GrabCut refinement is available (`method: grabcut`) so the two can be compared.

Where the mask is used. All colour and texture statistics are computed on leaf pixels only; shape and lesion features use the mask outline; the background perturbation replaces everything outside it. Segmentation is re-run on every perturbed image, so robustness numbers measure the whole pipeline and not only the classifier.

Known failure modes.

- Green backgrounds (grass, neighbouring plants, other leaves) pass the hue gate, so the mask can include them or the rule falls back to the whole image.
- Brown lesions touching the leaf edge are excluded from the mask, so their area is lost.
- Heavy shadow or glare can push value or saturation below the thresholds.
- A thin halo of original background can remain after background replacement.

What to report once the real run is done. `results/eda_segmentation_stats.json` gives the mean mask fraction and the share of images where segmentation fell back to the whole image; `results/eda_segmentation.png` shows 12 examples. Report the fallback rate in the paper, and look at the images before trusting it.

On our synthetic test images the first version labelled almost every image as all leaf (49% fell back); adding the hue gate fixed it. That says nothing about real photos.

6. Feature engineering

Each image becomes a vector of 105 hand-crafted features in three groups: 47 colour, 38 texture and 20 shape/edge/lesion, all computed on the leaf mask (`src/features.py`).

Group	Count	Features	Why it should help
Colour	47	Mean, standard deviation and skewness of each channel in RGB, HSV and LAB (27); 12-bin hue histogram; 8-bin saturation histogram	Chlorosis, necrosis and browning change hue and saturation; LAB separates lightness from colour

Group	Count	Features	Why it should help
Texture	38	Uniform LBP at two scales, (P=8, R=1) giving 10 bins and (P=16, R=2) giving 18 bins; GLCM contrast, dissimilarity, homogeneity, energy and correlation at distances 1 and 3, averaged over 4 angles (10), on 32 grey levels	Mould, spotting and blight patterns differ in surface texture
Shape, edge, lesion	20	Leaf area fraction, circularity, solidity, extent, eccentricity (5); 7 log-scaled Hu moments; Canny edge density, gradient mean and standard deviation, Laplacian variance (4); non-green lesion fraction, log lesion count, largest and mean lesion fraction (4)	Curl virus deforms leaves; spots differ in size and count

Lesion features. Inside the eroded leaf mask, a pixel counts as non-green when its excess-green index (2G - R - B over R + G + B) is below 0.15. Small regions (under 20 pixels) are dropped before counting.

Implementation notes.

- Features are extracted in parallel with joblib and cached as one matrix per split in manifest order, so they are computed once.
- Feature names start with the group (`color_`, `texture_`, `shape_`), so ablations are column selections.
- Non-finite values (rare) are replaced with 0 and reported.
- This stage is CPU-bound; a GPU does not help.

Course concepts to cite. Feature engineering and curse of dimensionality (105 features on roughly 580 training images), correlated features (RGB/HSV/LAB statistics overlap), and noise sensitivity of derived features.

7. Models and tuning protocol

Random Forest is the main model; an RBF SVM and a Logistic Regression act as a brief cross-check. Hyper-parameters are chosen on the validation split only, and the test split is used once, for final numbers (`src/models.py`, `scripts/04_baseline.py`).

Model	Settings	Searched grid	Course concept
Random Forest	300 trees, seed 42	max depth {none, 20, 40}, max features {sqrt, 0.3}, min leaf size {1, 2}	Bagging, variance reduction, bias-variance trade-off
SVM	RBF kernel, standardised inputs	C {1, 10, 100}, gamma {scale, 0.01, 0.001}	Margin maximisation, kernels
Logistic Regression	L2 penalty, multinomial, standardised inputs	C {0.01, 0.1, 1, 10}	Regularisation, linear decision boundary

Protocol.

1. Fit each configuration on the train split and score macro-F1 on the validation split; keep the best.
2. Refit the chosen configuration on train plus validation.
3. Evaluate once on the clean test split and report accuracy, macro-F1 with a 95% bootstrap interval, per-class precision, recall and F1, and the confusion matrix.
4. Report test metrics again on the subset of test images that have no near-duplicate or shared group in train or validation, when any are flagged.
5. Plot a Random Forest learning curve (train and validation macro-F1 against training-set size) to show where the bias-variance trade-off sits.

Why Random Forest as the main model. Hand-crafted features are heterogeneous, differently scaled and correlated; trees need no scaling, handle mixed feature types, give impurity importances and are cheap enough for low-end devices, which is the deployment setting in the synopsis.

Class balance. The synopsis argued class weighting was unnecessary because every class had 1,200 images. That argument applied to the augmented set. Per-class counts for the raw set are not yet known (the audit now prints them); if they are strongly imbalanced we will add class weights and keep macro-F1 as the headline metric.

8. Experiments: ablation and robustness

Two experiments reuse the same trained models: a feature-group ablation on clean data, and a robustness test that perturbs only the test images (`scripts/05_ablation.py`, `scripts/06_robustness.py`).

Ablation. Seven feature sets (colour, texture, shape, the three pairs, and all) crossed with the three models. Each model is refit on train plus validation using the hyper-parameters tuned on all features, and scored once on the clean test set. Reusing the all-feature hyper-parameters is a simplification; it slightly favours the full set.

Perturbations. 15 conditions, applied to test images only (`src/perturb.py`). Each is deterministic for a given seed, image and level.

Kind	Levels	Implementation	What it imitates
Brightness	x0.5, x0.7, x1.3, x1.6	Multiply pixel values, clip to 0-255	Dim light, strong sun
Contrast	x0.5, x0.7, x1.5, x2.0	Scale deviation from the image mean	Haze, flat or harsh light
Shadow	strength 0.3, 0.5, 0.7	Soft diagonal darkening with random direction and edge position	A shadow across the leaf
Background	grey, noise, soil, grass	Replace pixels outside the leaf mask with a synthetic texture, edge blurred	Different ground or plants behind the leaf

Robustness protocol.

1. Train every model once on clean train plus validation.
2. For each condition, perturb the test images, re-run segmentation, re-extract all features, and predict.
3. Report accuracy and macro-F1 per condition and the drop from the clean score (clean minus perturbed), per model and feature set. The Random Forest is evaluated on every feature set; SVM and Logistic Regression on all features.
4. Error analysis for the worst condition: confusion matrix, per-class F1 drop and the shift in predicted-class shares.

Why test-only perturbation. The question is how a model trained on the data as collected copes with new conditions, so training stays clean. Training on perturbed images is an optional follow-up, not part of the current design.

Caveat. Brightness and contrast are global; real fields mix lighting effects. Background replacement is only as good as the mask (section 5), and a failed mask means that image's background is left unchanged.

9. Explainability stability protocol

We treat explainability as something to measure: how much the feature ranking of the Random Forest changes between data folds and between clean and perturbed inputs (`src/explain.py`, `scripts/07_importance_stability.py`).

Importance measures.

- **Impurity importance** from the forest itself.
- **Permutation importance**: shuffle one feature (or one whole feature group) across the evaluation rows and record the drop in the mean probability the model gives to the true class.
- **Grouped permutation**: colour, texture and shape are each shuffled jointly. Colour statistics in RGB, HSV and LAB overlap, so single-feature rankings split credit arbitrarily among correlated features and are expected to be unstable; the group view asks a steadier question.

Two stability tests.

Test	Setup	Compared
Across folds	5-fold stratified cross-validation on train plus validation, with folds that keep each photo group together; one forest per fold	Rankings from every pair of folds
Across perturbations	The fixed forest from the baseline, importance recomputed on the clean test set and on each of the 15 perturbed test sets	Each perturbed ranking against the clean ranking

Agreement statistics. Spearman and Kendall rank correlation over all 105 features, and Jaccard overlap of the top-10 and top-20 features. For the three groups, the same statistics are reported over the group ranking.

Why the true-class probability, not F1. On a small test set most features cause no change in the predicted label when shuffled, so an F1 drop is exactly zero for many features and rank correlations become undefined or meaningless. The mean true-class probability changes smoothly, so the ranking has few ties. (This was found while testing on synthetic data, where F1-based importances produced undefined correlations.)

Interpretation caution. High rank agreement shows the model uses the same cues, not that those cues are medically meaningful. Low agreement under a perturbation shows the model's reasoning changed, which is the finding we want to report.

10. Evaluation metrics and statistical notes

Macro-F1 is the headline metric because every class counts equally; accuracy is reported only for comparison with earlier tomato-disease papers.

Metric	Definition or use	Why
Accuracy	Share of test images classified correctly	Comparability with prior work
Macro-F1	Mean of per-class F1	Easy classes cannot hide weak ones; missing a disease is costly
Per-class precision, recall, F1	From the test predictions	Shows which diseases fail
Confusion matrix	Row-normalised counts	Shows which diseases are mixed up and under which condition
Robustness drop	Clean score minus perturbed score, for accuracy and macro-F1	Direct measure of robustness
Importance stability	Spearman, Kendall, top-k Jaccard	Explainability as a number

Uncertainty. Test macro-F1 carries a 95% bootstrap interval (500 resamples of the test images). With a test set of roughly 125 images, about 20 per class if balanced, intervals will be wide: differences of a point or two between models or feature sets are not evidence of a real difference. The bootstrap resamples images and ignores the small group structure, which is acceptable because most groups contain a single image.

Not yet done. No significance test between models (a paired bootstrap or McNemar test on the same test images would be the natural choice), and a single split only. Repeating the grouped split with several seeds would show how much results depend on which images landed in the test set.

Leak-free reporting. Test metrics are also computed on the images with no near-duplicate or shared group in train or validation, and both numbers are reported when any image is flagged.

11. Problems encountered and how each was handled

Twelve problems so far; the first five come from the real dataset, the rest were found while testing the code on synthetic images and are listed because they shaped design decisions.

#	Problem	Seen on	Why it matters	Resolution
1	Synopsis says 7,200 images, 1,200 per class, fixed 840/240/120 split. The raw folder has 830 images	Real data	Test set is about 125 images; class balance unknown; synopsis claims need revising	Use the 830 originals, make our own grouped split, report the revision with reasoning
2	Augmented set: files are derived copies of source photos (shared file-name stem)	Real data (folder names and file names)	Siblings of one photo may sit in train and test, inflating scores; test images already carry brightness, flip and rotation changes	Do not use it for evaluation; use raw. Auditing it (`split_strategy: official`) remains possible

#	Problem	Seen on	Why it matters	Resolution
3	Raw set has no train/val/test split	Real data	We must create one without leakage	Grouped 70/15/15 split per class, seed 42, manifest committed
4	Random file names, so copies cannot be grouped by name	Real data	45 of 830 images (5.4%) are near-copies; 9 pairs cross classes	Perceptual-hash candidates verified by thumbnail correlation (section 4); cross-class pairs flagged for review
5	Folder spelling differs between folders (`septora` vs `Septoria`, `Early_Blight` vs `Early_blight_leaf`)	Real data	A class would be silently skipped	Class matcher accepts both spellings; the audit lists unmatched folders
6	First near-duplicate hash (difference hash) scored identical images 28-41 bits apart after recompression	Synthetic	Copies would be missed	Switched to a DCT perceptual hash
7	The coarse hash also matched different leaves, risking chains that merge many images into one group	Synthetic	A giant group would land in one split and unbalance it	Added a thumbnail-correlation check (at least 0.98) and a warning when one group exceeds 2% of the data
8	First segmentation labelled saturated brown soil as leaf (49% whole-image fallback)	Synthetic	Features and background replacement would be meaningless	Added a hue gate (20-100) and a larger closing kernel
9	F1-based permutation importance gave many exact zeros, so rank correlations were undefined	Synthetic	Stability statistic would be meaningless	Importance measured as drop in true-class probability

#	Problem	Seen on	Why it matters	Resolution
10	Ordinary cross-validation would put copies of a photo in different folds	Design review	Optimistic scores and inflated importance stability	Folds keep each photo group together
11	Reading thousands of files straight from Google Drive is slow	Colab setup	Wasted GPU-free session time	Unzip the dataset to the notebook's local disk first
12	Colab deletes local files when a session ends	Colab setup	Lost cache, features and models	Notebook backs `work/` up to Drive and restores it

Still open. The 9 cross-class pairs need visual inspection; the hash thresholds need checking on real photos; per-class counts of the raw set are not yet known.

12. Limitations and threats to validity

The main limits are a small dataset, an imperfect leaf mask, and grouping that catches copies but not repeat shots of the same plant. Each is stated so the paper's conclusions are not stretched beyond them.

Data

- **Small sample.** About 830 photos (785 independent) in six classes. The test set has roughly 125 images, so every score has a wide interval, and per-class results rest on about 20 images.
- **Class balance unknown.** Raw per-class counts have not been checked; imbalance would affect macro-F1 and may require class weights.
- **Single split.** One grouped split with seed 42. Results depend on which images landed in test; repeated splits would quantify this.
- **Same-plant leakage not removed.** Grouping catches resized, recompressed, flipped and re-lit copies. It does not catch a different photo of the same leaf or plant (for example burst shots with close timestamps), so test images may be easier than a truly new field. Grouping by capture time, where file names carry it, is a possible refinement.

- **Possible label noise.** 9 near-duplicate pairs carry two labels; the labels were assigned manually from visible symptoms, and some diseases look alike.
- **Provenance.** Some file names suggest sources other than a phone in the authors' fields. Not verified.
- **Resizing.** Images are squashed to 256x256 without keeping aspect ratio, identically in every split. Fine texture is lost at this size.

Method

- **Segmentation is unsupervised and approximate.** It works against soil but not against green backgrounds, and it drops brown lesions on the leaf edge. Shape and lesion features and the background perturbation inherit these errors.
- **Synthetic perturbations.** Brightness, contrast, shadow and background replacement are global or single-pattern; real fields combine effects. Background replacement leaves a thin halo and does nothing where segmentation failed.
- **Test-only perturbation.** We do not train on perturbed images, so the results describe a model trained on data as collected. They do not show the best achievable robustness.
- **Hyper-parameters tuned on all features** are reused in the ablation and robustness tests, which slightly favours the full feature set.
- **Hash thresholds** were set on synthetic images and need checking on the real photos.
- **Correlated features.** Colour statistics overlap, so single-feature importance is unstable by construction; the grouped view is the safer reading.
- **No significance tests** between models or conditions yet.

Scope

- One crop, one country, one dataset. Conclusions do not transfer to other crops or regions without testing. The PlantCity out-of-distribution test mentioned in the synopsis is optional and not yet done.
- Classical features and models only, by design. A CNN reference would show what is traded away, but is not part of the current plan.

13. Results (to fill from real runs)

No modelling results exist on the real data yet; the only results so far are the audit numbers in section 3. Every number below must be copied from the listed file, never typed from memory or taken from the synthetic test runs.

Experiment	Source files in `results/`	What to report	Status
Dataset audit	`dataset_audit.json`, `grouping_summary.json`	Image count, groups, copies, cross-class pairs, per-class counts	Done on Oct 13 (per-class counts missing)
EDA	`eda_samples.png`, `eda_class_stats.csv`, `eda_brightness_by_class.png`, `eda_image_sizes.csv`	Sample images, brightness and saturation per class (illumination shortcut check), original image sizes (uniform-resolution claim)	Pending
Segmentation check	`eda_segmentation_stats.json`, `eda_segmentation.png`	Mean mask fraction, whole-image fallback rate	Pending
Near-duplicates	`duplicates_summary.json`, `duplicate_examples.png`, `test_leakage_flags.csv`	Pairs, cross-split pairs, share of test images flagged	Pending
Baseline	`baseline_test_metrics.csv`, `baseline_per_class_test.csv`, `confusion_*_test.png`, `baseline_tuning_*.csv`, `learning_curve_rf.csv`	Accuracy, macro-F1 with 95% interval for RF, SVM, LR; per-class F1; best parameters; leak-free scores	Pending
Ablation	`ablation.csv`, `fig_ablation.png`	Macro-F1 for 7 feature sets x 3 models	Pending

Experiment	Source files in `results/`	What to report	Status
Robustness	`robustness.csv`, `robustness_drop.csv`, `fig_robustness_curves.png`, `fig_robustness_drop_heatmap.png`	Accuracy and macro-F1 drop per condition, model and feature set	Pending
Importance stability	`importance_stability_cv.json`, `importance_stability_perturbed.csv`, `importance_top_features.csv`, `fig_group_importance.png`, `fig_importance_stability.png`	Rank agreement across folds and perturbations; top features; group importance	Pending
Error analysis	`error_pairs.csv`, `misclassified_examples.png`, `confusion_rf_worst_condition.png`, `per_class_f1_drop_worst.csv`	Most-confused class pairs, worst condition, per-class drop	Pending

Questions each result should answer in the write-up.

1. Does the clean score drop when the 125-image test set replaces a leaky or augmented one? Compare the all-test and leak-free scores.
2. Which perturbation family hurts most, and which classes lose most?
3. Does a single feature group carry the model, and does that group collapse under a particular perturbation?
4. Is the importance ranking stable across folds, and does it survive perturbations? Report both the per-feature and the grouped view.
5. Which diseases are confused with which, and does the confusion change under perturbation?

14. Reproducibility, paper mapping and open items

Every experiment is reproducible from one repository, one config file and one committed split; this section also maps the log onto the report and paper sections and lists what remains.

Reproducibility.

- **Code:** `github.com/manahilcpkhan/tomato-robust-ml`. Logic lives in `src/`, one runnable script per stage in `scripts/` (00 audit through 09 figures), a thin Colab runner in `notebooks/`.
- **One config:** `config.yaml` holds the seed (42), paths, feature settings, search grids, perturbation levels and thresholds. Personal paths go in a git-ignored override.
- **One split:** `data/manifest.csv`, committed once.
- **Large files** (image cache, features, models) stay out of Git and are backed up to Drive; small results are committed.
- **Software:** Python with NumPy, pandas, SciPy, scikit-learn, scikit-image, OpenCV, joblib and matplotlib (`requirements.txt`). CPU only; no GPU is used.
- **Work split:** Member A data and perturbation, Member B features and models, Member C evaluation, explainability and report integration.

Where each part goes.

This log	Mid-semester report (Nov 13)	Final paper (Dec 24)
1, 2	Introduction	Introduction, dataset
3, 11	Challenges and plan revisions	Dataset section, limitations
4, 5, 6, 7	Methodology (tie each choice to a course concept)	Method
8, 9, 10	Methodology, experiment design	Experiments
13	Preliminary results	Results and error analysis
12	Challenges	Limitations

Not covered here: the literature survey. Both the synopsis and the mid-semester report are graded on it. The synopsis cites four works (PlantVillage, PlantDoc, the Pakistan tomato dataset, PlantCity). A fuller survey should cover lab versus field datasets, handcrafted-feature and classical-ML plant disease work, robustness and distribution shift in plant disease recognition, and feature-importance stability. We have not collected or checked any further references.

Open items.

- [ ] Run stage 01 (manifest) and commit `data/manifest.csv` and `results/grouping_summary.json`.
- [ ] Read the per-class counts from the updated audit and decide on class weights.

- [ ] Inspect the 9 cross-class near-duplicate pairs and the example pairs from stage 02.
- [ ] Check `eda_image_sizes.csv` against the uniform-resolution claim and look at EXIF data for the provenance question.
- [ ] Check the segmentation fallback rate and the 12 example masks; try GrabCut if it is high.
- [ ] Run stages 03 to 09 and fill section 13.
- [ ] Audit `Augmented Dataset` once to quantify its leakage and confirm its 7,200 count (a single paragraph for the paper).
- [ ] Write the literature survey.
- [ ] Decide whether to add repeated splits, significance tests, or training on augmented copies of training photos only.